

Laboration 7 – TLS, autentisering och hemligheter

Uppgift

Kör ett lokalt MQTTS-flöde, identifiera hur klienten verifierar brokern och dokumentera hur motsvarande kommunikationsväg i ert IoT-system ska skyddas. Git är valfritt för den fristående laborationen. IoT-caset fortsätter använda Git.

Del 1 – Rita säkerhetslagren

Rita klient, nätverk och tjänst. Markera:

- Var TLS börjar och slutar
- Vem som kontrollerar servercertifikatet
- Var klientens autentiseringsuppgift kontrolleras
- Vilken åtkomst en godkänd klient får

Del 2 – Kör grunddemot

Demot använder två processroller men samlar körningen i en terminal:

- Gatewayen kör MQTT-brokern
- Sensornoden publicerar telemetri
- Bearbetningstjänsten prenumererar på telemetrin

```
cd dag_7/demo_mqtts
bash run_demo.sh
```

Fyll i [testprotokollet](#). Ange varför testet med fel CA stoppas före MQTT och varför rätt CA låter klienten verifiera brokern.

Del 3 – Undersök certifikatet

Här blev jag tvungen att ändra formateringen för att få texten liten nog att få plats på en rad. Annars kan det tolkas som flera kommandon.

```
openssl x509 -in generated/server.crt -noout -subject -issuer -dates -ext subjectAltName
```

Besvara:

- Vilket namn får certifikatet användas för?
- Vem har utfärdat certifikatet?
- När slutar certifikatet gälla?
- Varför behöver klienten `generated/ca.crt`?

Del 4 – Testa en avgränsad ändring

Kör klienten med betrodd CA men fel nyckel:

```
python3 https_client.py --trust-ca --api-key felaktig
```

Förväntat svar är HTTP 401 när HTTPS-servern kör med en annan demo-nyckel. Förklara varför TLS ändå har fungerat.

Kör därefter [HTTPS-demot](#) som jämförelse om ni ska skilja TLS-verifiering från en API-nyckelkontroll. Använd endast demohemligheten.

Del 5 – Hotmodellera

Välj en kommunikationsväg i kurscaset och fyll i minst tre hot i mallen. Minst ett hot ska röra en falsk motpart och minst ett ska röra läckta eller felaktiga autentiseringsuppgifter.

För varje motåtgärd ska ni ange ett test. Formuleringen "vi använder TLS" räcker inte utan en beskrivning av vad klienten verifierar och hur ett fel syns.

Valbar fördjupning

Välj en:

- Skapa ett nytt servercertifikat vars SAN inte innehåller `localhost` och observera värddamnnsfelet
- Föreslå olika behörighet för sensor, konsument och driftklient
- Beskriv hur ömsesidig TLS skulle låta servern verifiera ett klientcertifikat

Arbetsresultat

- Ett godkänt HTTPS-anrop
- Minst två negativa test med förväntat resultat
- Ifylld hotmodell och testprotokoll
- En konkret plan för hemlighetshantering
- En förklaring av vilka certifikat som får finnas i ESP32-firmware och vilken privat nyckel som måste skyddas
- Tydligt angivna begränsningar i labbmiljön

Fortsätt därefter med [IoT-caset](#).